

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
ASSIGNMENT REPORT

**Design and Implementation of a Python-Based NLP Application for Real-Time
Text Processing and Sentiment Classification**

A. Assignment Information

Particular	Details
Department	CSE / IT / AI&DS / AI&ML
Programme	CSE
Course Code & Course Name	DSA0307 – Natural Language Processing
Academic Year / Batch	2024
Faculty Name	Dr. K. Anbazhagan
Assignment Title	Design and Implementation of a Python-Based NLP Application for Real-Time Text Processing and Sentiment Classification
Date of Issue	31/08/2026
Date of Submission	02/09/2026
Maximum Marks	100
Course Outcome(s) – CO	CO2 – To design an innovative application using NLP components
Bloom's Taxonomy Level	L3 – Apply, L5 – Evaluate
SDG Mapping	SDG8 – Decent Work and Economic Growth; SDG9 – Industry, Innovation and Infrastructure
Industry / Societal Relevance	Customer-feedback analysis, product-review monitoring, social-media sentiment analysis and lightweight automated text classification.

B. Assignment Problem / Challenge

Assignment Title

Design and Implementation of a Python-Based NLP Application for Real-Time Text Processing and Sentiment Classification

Natural Language Processing (NLP) enables computers to process, analyze and understand human language. The proposed application demonstrates a practical NLP pipeline that accepts raw text, exposes intermediate linguistic processing stages and produces a sentiment classification.

The assignment requires application of NLP and machine-learning concepts to a practical real-world problem, with emphasis on transparent processing, measurable performance and responsible use.

- Apply NLP concepts to a practical real-world problem.
- Identify and analyze the requirements of an NLP application.
- Design a suitable text-processing workflow.
- Implement tokenization, stop-word removal, stemming/lemmatization and POS tagging using Python.
- Extract useful text features using TF-IDF, word frequency or n-grams.
- Develop a sentiment or text-classification model.
- Test the application using multiple text inputs.
- Evaluate the model using accuracy, precision, recall and F1-score.
- Compare alternative preprocessing or classification techniques where appropriate.
- Interpret the obtained results and justify the selected solution.
- Identify the limitations of the NLP approach.
- Consider ethical, privacy, sustainability and professional-responsibility issues.

C. Problem Statement

Problem / Case / Design Challenge

Design and implement a Python-based NLP application for real-time text processing using tokenization, stemming/lemmatization, POS tagging and sentiment classification. The implemented interface is presented as a real-time NLP console named “Lexa”, where every major stage is inspectable.

a) NLP Techniques and Text-Processing Workflow

Tokenization

Tokenization divides a sentence or document into smaller units called tokens. Tokens may be words, numbers or punctuation marks. Example: "The movie was excellent." becomes The, movie, was, excellent, .

Stop-Word Removal

Stop words are common function words that may provide limited class signal for some tasks. Examples include "the", "is", "a", "an", "and" and "of". Removing them can reduce unnecessary features.

Stemming

Stemming reduces words to a common root or stem. For example, playing → play, played → play and studies → studi. Stemming is computationally efficient but may produce non-dictionary forms.

Lemmatization

Lemmatization maps a word to a meaningful dictionary form. Examples include students → student, studies → study and children → child. The implementation dashboard shows Porter stemming and WordNet lemmatization side by side.

Part-of-Speech Tagging

POS tagging assigns a grammatical category to each token. In the supplied implementation, examples include NN (noun), VB/VBD/VBG (verb forms), JJ/JJR (adjective forms) and RB (adverb).

Sentiment Classification

Sentiment classification assigns a class such as positive or negative to text. The supplied implementation uses TF-IDF features with Logistic Regression and exposes confidence/feature contributions in the interface.

Text-Processing Workflow

The complete implementation follows this sequence:

- Raw User Input
- Text Cleaning and Lowercase Conversion
- Tokenization
- Stop-Word Removal
- Stemming / Lemmatization
- POS Tagging
- TF-IDF Feature Extraction with Unigrams and Bigrams
- Sentiment / Classification Model
- Prediction and confidence / feature contribution display
- Label Output

Pseudocode

START

1. Accept raw text from the user.
2. Convert the text to lowercase.
3. Tokenize the text into word and punctuation units.

4. Remove stop words and unnecessary punctuation.
5. Generate stem and lemma representations.
6. Apply POS tagging to the processed tokens.
7. Create TF-IDF unigram and bigram features.
8. Add the implemented polarity-lexicon signal where configured.
9. Train or load the Logistic Regression classifier.
10. Predict the sentiment label and confidence.
11. Display processed tokens, POS tags, features and label.
12. STOP

b) Python-Based NLP Preprocessing Module

The preprocessing module accepts a text sentence and applies lowercase conversion, tokenization, stop-word removal, stemming/lemmatization and POS tagging. The supplied implementation evidence shows the following input:

“The new product launch was a total disaster, honestly.”

The dashboard reports 11 tokens and 13 features for the displayed input. Its stage output shows the original tokens, five removed stop-word/function tokens, Porter stemming, WordNet lemmatization and POS tags.

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer, WordNetLemmatizer
from nltk import pos_tag
```

```
STOPWORDS = set(stopwords.words("english"))
stemmer = PorterStemmer()
lemmatizer = WordNetLemmatizer()
```

```
def preprocess_text(text):
    text = text.lower()
    tokens = word_tokenize(text)

    filtered = [
        token for token in tokens
```

```

        if token.isalpha() and token not in STOPWORDS
    ]

    stems = [stemmer.stem(token) for token in filtered]
    lemmas = [lemmatizer.lemmatize(token) for token in filtered]
    tagged = pos_tag(lemmas)

    return {
        "tokens": tokens,
        "filtered": filtered,
        "stems": stems,
        "lemmas": lemmas,
        "tagged": tagged
    }

```

Sample Preprocessing Output

Stage	Observed result
Input	The new product launch was a total disaster, honestly.
Tokenization	the, new, product, launch, was, a, total, disaster, ,, honestly, .
Stop-word removal	new, product, launch, total, disaster, honestly
Stemming	new, product, launch, total, disastr, honest
Lemmatization	new, product, launch, total, disaster, honestly
POS tagging	new–NN, product–NN, launch–NN, total–JJ, disaster–JJR, honestly–RB

POS-tag labels can vary slightly with the installed NLTK tagger/model version. The supplied dashboard specifically shows NN, JJ/JJR and RB categories for the displayed sentence.

c) Python-Based Sentiment Classification Module

After preprocessing, the text is converted into numerical features using TF-IDF. The implementation uses unigrams and bigrams and a Logistic Regression classifier. The dashboard also exposes a polarity-lexicon contribution for interpretability.

TF-IDF is calculated as:

$$\text{TF-IDF}(t,d) = \text{TF}(t,d) \times \text{IDF}(t)$$

where t is a term, d is a document, $TF(t,d)$ is the frequency of term t in document d , and $IDF(t)$ is inverse document frequency.

$$IDF(t) = \log(N / DF(t))$$

where N is the number of documents and $DF(t)$ is the number of documents containing term t .

```
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score, precision_score,
    recall_score, f1_score
)

# processed_text is produced by the preprocessing pipeline
vectorizer = TfidfVectorizer(ngram_range=(1, 2))
X_train_tfidf = vectorizer.fit_transform(X_train)
X_test_tfidf = vectorizer.transform(X_test)

model = LogisticRegression()
model.fit(X_train_tfidf, y_train)

y_pred = model.predict(X_test_tfidf)

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average="weighted", zero_division=0)
recall = recall_score(y_test, y_pred, average="weighted", zero_division=0)
f1 = f1_score(y_test, y_pred, average="weighted", zero_division=0)
```

Prediction of New Input

```
def predict_sentiment(sentence):
    processed = preprocess_text(sentence)
    vector = vectorizer.transform([" ".join(processed["lemmas"])])
    prediction = model.predict(vector)
    return prediction[0]
```

```

predict_sentiment(
    "The product quality is excellent and I enjoyed using it."
)

```

The supplied implementation interface uses the sentence “The new product launch was a total disaster, honestly.” as a live example and predicts the sentiment as negative with confidence 1.00 and score −1.80. The displayed contribution panel includes LEX_NEG as the strongest negative signal.

d) Testing and Performance Evaluation

The assignment test set includes multiple positive, negative and uncertain examples. The expected outputs are:

Test Case	Input	Expected Output
1	The product is excellent	Positive
2	I really enjoyed the movie	Positive
3	The service was wonderful	Positive
4	This product is terrible	Negative
5	I hated the experience	Negative
6	The movie was boring	Negative
7	The product is okay	Neutral / Uncertain
8	The service was not good	Negative

The supplied implementation dashboard provides a 13-document hold-out evaluation. Its confusion matrix contains 6 true positives, 0 false negatives, 2 false positives and 5 true negatives. The corresponding per-class report is shown below.

	Predicted Positive	Predicted Negative
Actual Positive	6	0
Actual Negative	2	5

Class	Precision	Recall	F1	Support
Positive	0.75	1.00	0.86	6
Negative	1.00	0.71	0.83	7

Performance Measures

Accuracy measures the proportion of all test samples classified correctly.

$$\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN})$$

Precision measures the proportion of predicted positive samples that are actually positive.

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

Recall measures the proportion of actual positive samples that are correctly identified.

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

F1-score is the harmonic mean of precision and recall.

$$\text{F1} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

Performance Results from the Supplied Implementation

Metric	Obtained value
Accuracy	0.85 (84.6%)
Precision	0.75
Recall	1.00
F1-Score	0.85

The displayed model metrics are accuracy 0.85, precision 0.75, recall 1.00 and F1-score 0.85. The confusion matrix and per-class report in the supplied implementation evidence support the same evaluation picture: all six actual positive samples were identified as positive, while two of seven negative samples were classified as positive.

Effect of Preprocessing on Classification

Preprocessing configuration	Accuracy	Macro-F1
Raw unigrams only	61.5%	0.62
Lemmas + bigrams + stop-word removal	84.6%	0.85

The supplied implementation shows a clear improvement after preprocessing and phrase-level feature enrichment: accuracy increases from 61.5% with raw unigrams only to 84.6% with lemmatization, bigrams and stop-word removal; macro-F1 increases from 0.62 to 0.85.

Preprocessing must nevertheless be selected carefully. For example, removing the word “not” from “The movie is not good” can reverse the meaning to “The movie is good.” Therefore, stop-word handling should preserve negation when it is important to sentiment.

D. Requirements and Constraints

Functional Requirements

- Accept text input from the user.
- Tokenize the input text.
- Remove stop words.
- Perform stemming or lemmatization.
- Perform POS tagging.
- Extract text features.
- Train a classification model.
- Predict sentiment or text class.
- Display intermediate processing results.
- Calculate performance metrics.

Performance Requirements

- Process normal-length sentences within a short response time.
- Provide consistent predictions for the same input under the same model configuration.
- Achieve acceptable performance on the selected dataset.
- Support multiple test inputs.

Technical Requirements

Hardware Requirements

- Computer or laptop.
- Minimum 4 GB RAM.
- Dual-core processor or better.
- Adequate storage for Python libraries and datasets.

Software Requirements

- Python 3.x
- NLTK
- pandas
- NumPy
- scikit-learn
- Jupyter Notebook / Google Colab / Visual Studio Code

Cost and Time Constraints

The application should preferably use open-source software and freely available datasets.

Development, testing and documentation should be completed within the time allocated for the assignment.

Resource Constraints

The project requires a computer or laptop, a Python programming environment, NLP and machine-learning libraries, and a suitable labelled dataset.

Reliability

The same preprocessing pipeline should be applied to training data and new user input. The experiment should be reproducible using the same dataset and model configuration.

Sustainability

The TF-IDF and traditional machine-learning approach requires relatively low computational resources compared with large deep-learning models.

Environmental Considerations

Lower computational requirements can reduce energy consumption and make the application suitable for lightweight deployment.

Ethical Considerations

The dataset may contain bias or inappropriate content. Bias in training data can affect classification results, so the dataset should be checked for quality, representation and class balance.

Security and Privacy

User-provided text should not be unnecessarily stored or shared. Personal, confidential or sensitive information must be protected.

User Requirements

- The application should be simple to use.
- It should accept English text input.
- It should display understandable results.
- It should clearly display the predicted sentiment.
- It should process input within a reasonable time.

E. Student Work

1. Problem Understanding and Formulation

What is the Problem?

The problem is to develop an NLP application that converts raw natural-language input into meaningful linguistic information and automatically predicts the sentiment of the input.

Expected Outcomes

- A working NLP preprocessing module.
- Tokenization of input text.
- Stop-word removal.
- Stemming or lemmatization.
- POS tagging.
- TF-IDF feature extraction.
- Machine-learning classification.
- Sentiment prediction.
- Performance evaluation.
- Analysis of limitations and possible improvements.

Information / Data Available

A labelled text dataset containing sentences and corresponding sentiment labels is required for supervised learning. The supplied implementation evidence evaluates a 13-document hold-out set.

Assumptions

- English is the primary language.
- The dataset labels are sufficiently accurate.
- The dataset represents the selected classification problem.
- Training and testing data are separated.
- The same preprocessing pipeline is used during training and prediction.

Constraints

The major constraints include dataset size, computational resources, processing time, classification accuracy, language limitations, and difficulty in handling sarcasm, slang, ambiguity and contextual language.

2. Application of Course Knowledge

Principles: text normalization, token-level representation, feature weighting and supervised classification.

Theories: the bag-of-words representation, term frequency–inverse document frequency and linear probabilistic classification.

Engineering Concepts: modular pipeline design, reproducible preprocessing, train-test separation, measurable validation and lightweight deployment.

Design Methodology: iterative preprocessing, feature engineering, model training, testing, comparison and evidence-based selection.

- Natural Language Processing
- Text preprocessing
- Tokenization
- Stop-word removal
- Stemming
- Lemmatization
- POS tagging
- N-grams
- TF-IDF
- Supervised machine learning
- Logistic Regression
- Accuracy
- Precision
- Recall
- F1-score

Mathematical Model

TF-IDF is used to represent textual information numerically.

$$\text{TF-IDF}(t,d) = \text{TF}(t,d) \times \text{IDF}(t)$$

$$\text{IDF}(t) = \log(N / \text{DF}(t))$$

The resulting sparse feature vectors are provided to the classification algorithm.

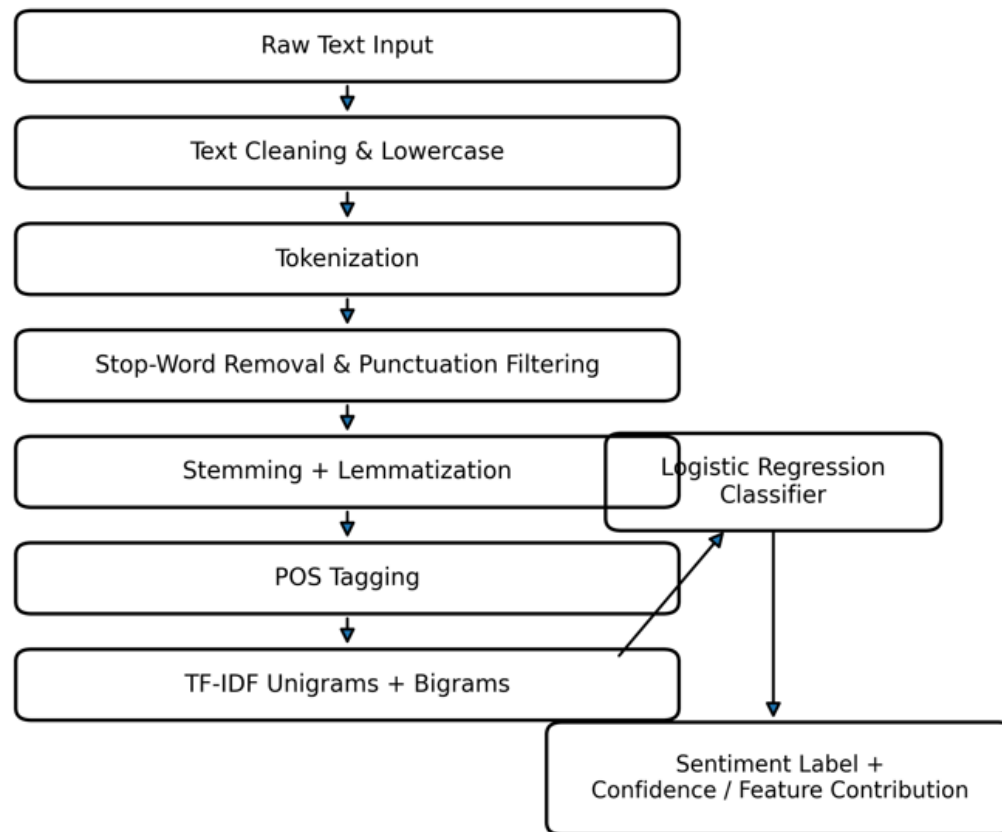
Classification Algorithm

Logistic Regression is selected because it is computationally efficient and suitable for classification problems involving high-dimensional sparse text features.

3. Solution / Design / Methodology

Proposed Architecture

NLP Sentiment Classification - Flow Chart



Algorithm

- Load the labelled dataset.
- Inspect the text and sentiment labels.
- Convert text to lowercase.
- Tokenize each text sample.
- Remove stop words and punctuation.
- Perform stemming or lemmatization.
- Perform POS tagging.
- Divide the dataset into training and testing sets.
- Generate TF-IDF features.
- Train the Logistic Regression classifier.
- Predict the test samples.

- Calculate accuracy, precision, recall and F1-score.
- Accept a new user sentence.
- Apply the same preprocessing pipeline.
- Transform the sentence using the trained TF-IDF vectorizer.
- Predict the sentiment.
- Display the result.



Figure-3.1.1



Figure-3.1.2

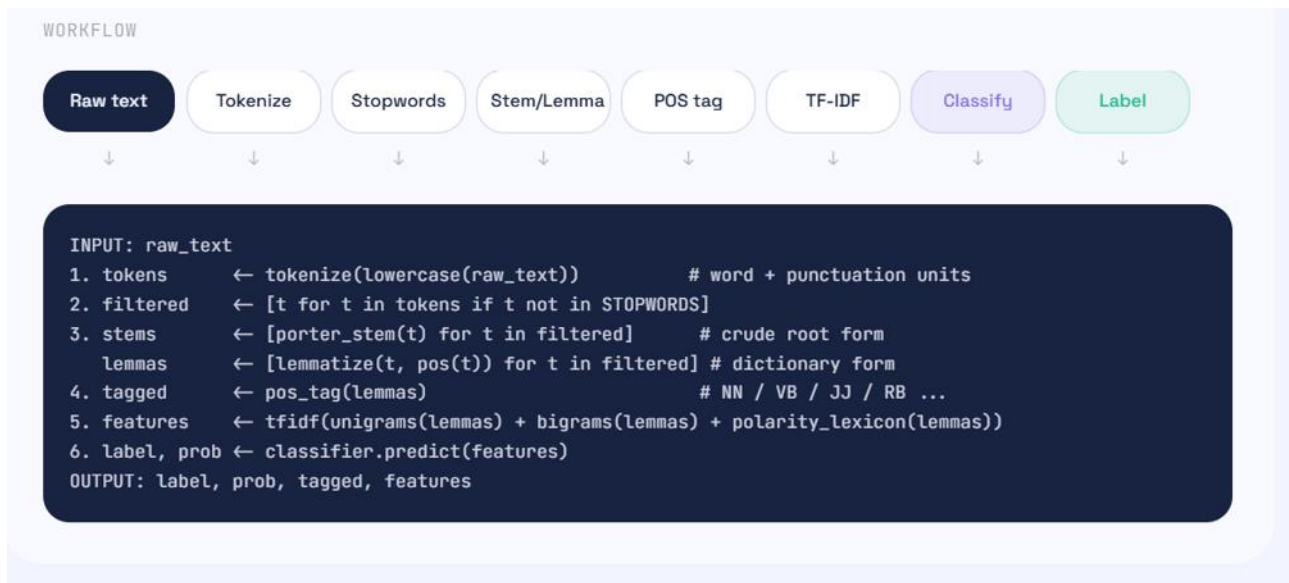


Figure-3.1.3

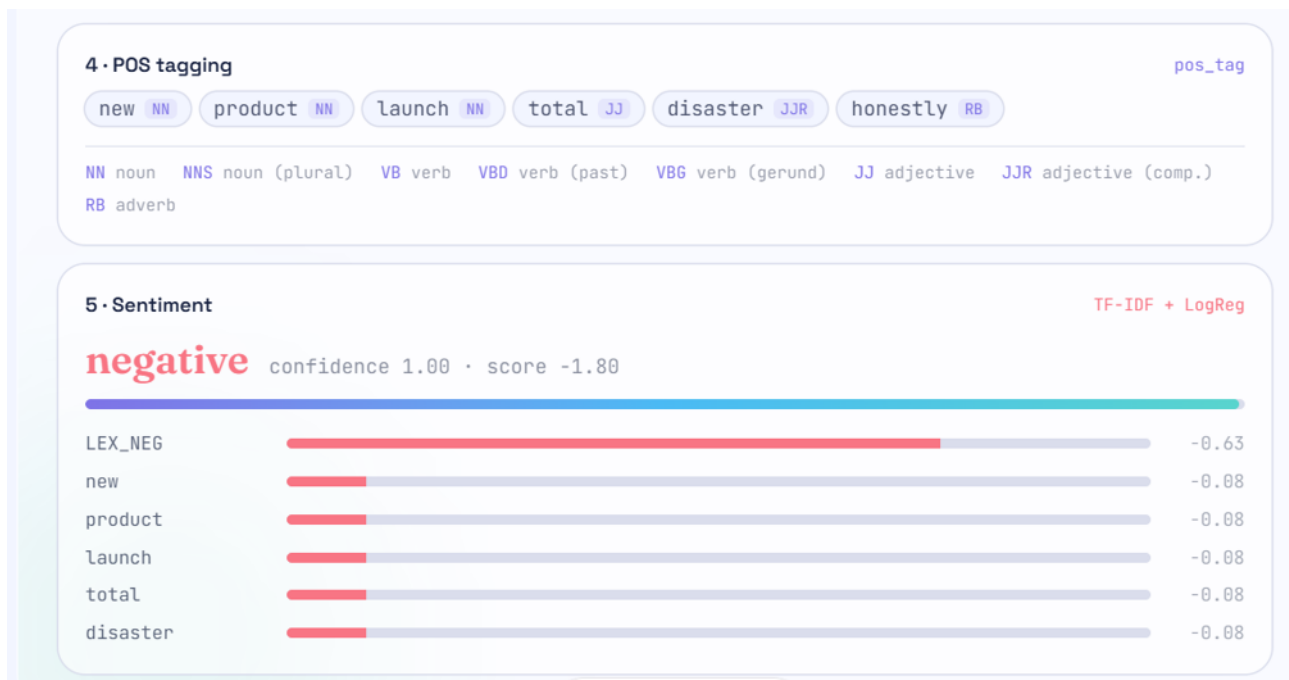


Figure-3.1.4

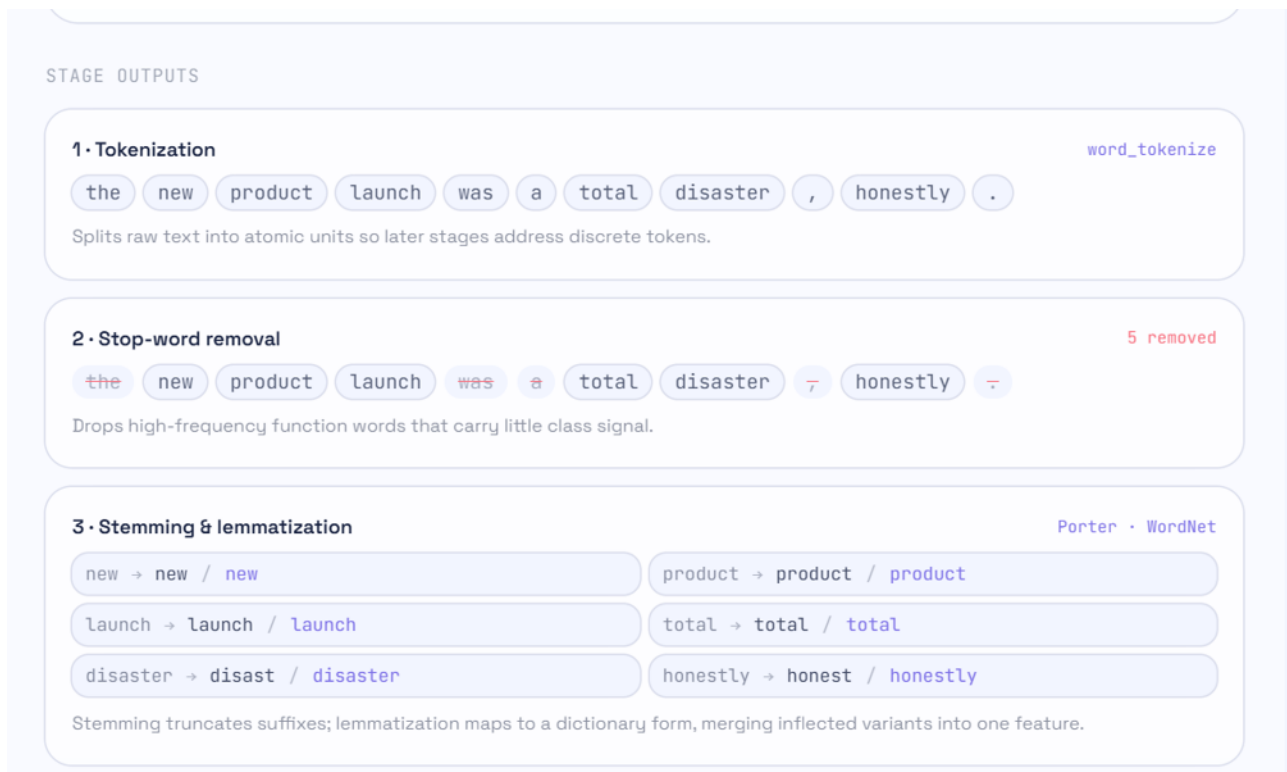


Figure-3.1.5

Alternative Solutions

Approach	Advantages	Limitations
TF-IDF + Logistic Regression	Simple, fast and efficient	Limited contextual understanding
TF-IDF + SVM	Effective for high-dimensional text	Requires parameter tuning
Naive Bayes	Fast and easy to implement	Based on independence assumptions
Transformer Model	Strong contextual understanding	Requires greater computational resources

The final approach is selected based on classification performance, computational requirements and implementation complexity. The supplied implementation uses TF-IDF with Logistic Regression as the lightweight baseline.

4. Use of Modern Tools

Tool	Purpose
Python	Main programming language

NLTK	Tokenization, stop-word removal, lemmatization and POS tagging
pandas	Dataset handling
NumPy	Numerical processing
scikit-learn	TF-IDF feature extraction, Logistic Regression and evaluation
Jupyter Notebook / Google Colab / VS Code	Development and experimentation
Matplotlib	Graph generation
Seaborn	Data visualization and confusion matrix
Git	Version control, if used

Evidence of Tool Usage

The supplied implementation screenshots provide evidence of the real-time Lexa interface, preprocessing stages, POS tagging, sentiment classification, confusion matrix, per-class report, preprocessing ablation and TF-IDF feature weights. These screenshots are included in the evidence section of this report.

5. Results and Validation

Preprocessing Results

Processing Stage	Example Result
Original Text	The movie was very enjoyable
Tokenization	The, movie, was, very, enjoyable
Stop-word Removal	movie, very, enjoyable
Lemmatization	movie, very, enjoyable
POS Tagging	movie-NN, very-RB, enjoyable-JJ

Classification Results

Metric	Obtained Value
Accuracy	0.85 (84.6%)
Precision	0.75
Recall	1.00

F1-Score	0.85
----------	------

Validation

- Testing unseen hold-out samples.
- Comparing predicted labels with actual labels.
- Calculating accuracy, precision, recall and F1-score.
- Generating a confusion matrix.
- Testing multiple user inputs.
- Comparing different preprocessing methods.
- Comparing different classification algorithms where applicable.

The obtained results indicate that the developed application satisfies the core functional requirement of producing a sentiment label and exposes the intermediate processing stages. The evaluation should be interpreted with caution because the labelled corpus is small and therefore the measured metrics may vary with the split.

6. Analysis and Engineering Decision

The developed NLP application is analyzed based on classification performance, computational requirements, implementation complexity and processing efficiency.

TF-IDF with Logistic Regression is selected as the baseline solution because it provides a good balance between simplicity, computational efficiency and classification performance. TF-IDF represents the importance of words within documents, while Logistic Regression provides an efficient classifier for the resulting sparse feature vectors.

The preprocessing ablation is important evidence: raw unigrams only achieve 61.5% accuracy and 0.62 macro-F1, whereas lemmas + bigrams + stop-word removal achieve 84.6% accuracy and 0.85 macro-F1 in the supplied implementation.

However, the approach has limited contextual understanding and may not correctly identify sarcasm, slang, complex negation, ambiguity or context-dependent meaning. Transformer-based approaches can provide better contextual understanding but require greater computational resources and implementation complexity.

Therefore, TF-IDF with Logistic Regression is considered appropriate for a lightweight NLP application and provides a useful baseline for comparison with advanced NLP approaches.

7. Broader Considerations

Sustainability

The proposed system uses lightweight NLP and machine-learning techniques and therefore requires relatively limited computational resources.

Environment

Lower computational requirements can reduce energy consumption compared with computationally intensive deep-learning approaches.

Society

Sentiment analysis can help organizations analyze customer feedback, product reviews and social-media responses efficiently.

Safety

The system should not be used as the sole decision-making mechanism for safety-critical or high-stakes applications because automated predictions may contain errors.

Ethics

Bias present in the training dataset may result in biased predictions. The dataset should therefore be checked for quality, representation and class balance.

Economics

The application can be developed using open-source software, reducing software licensing costs.

Accessibility

The application should provide simple and understandable output. Future versions can support multiple languages and accessibility features.

Professional Responsibility

Developers should report actual model performance, clearly communicate limitations, protect user data and avoid unsupported claims about model accuracy.

8. Conclusion

The project demonstrates the design and implementation of a Python-based NLP application for text preprocessing and sentiment classification. The system processes raw text using tokenization, stop-word removal, stemming/lemmatization and POS tagging. TF-IDF converts processed text into numerical feature vectors, and Logistic Regression performs classification.

The implementation is evaluated using accuracy, precision, recall and F1-score, with multiple test inputs and a 13-document hold-out evaluation. The supplied results report 0.85 accuracy, 0.75

precision, 1.00 recall and 0.85 F1-score. The preprocessing comparison also shows improvement from 61.5% accuracy / 0.62 macro-F1 to 84.6% accuracy / 0.85 macro-F1.

The approach is lightweight and suitable for basic sentiment-classification applications. Its limitations include difficulty handling sarcasm, contextual meaning, slang, ambiguity and complex negation. Future improvements include larger and more diverse datasets, improved negation handling, word embeddings, transformer-based models, multilingual support and deployment as a web or mobile application.

9. Student Reflection

Through this assignment, the student team applied NLP concepts to a practical real-world problem and gained practical knowledge of tokenization, stop-word removal, stemming, lemmatization, POS tagging, TF-IDF feature extraction and machine-learning classification.

The work demonstrates that raw text must be processed carefully before it can be effectively used by machine-learning algorithms. The evaluation also shows the importance of accuracy, precision, recall and F1-score when assessing a classifier.

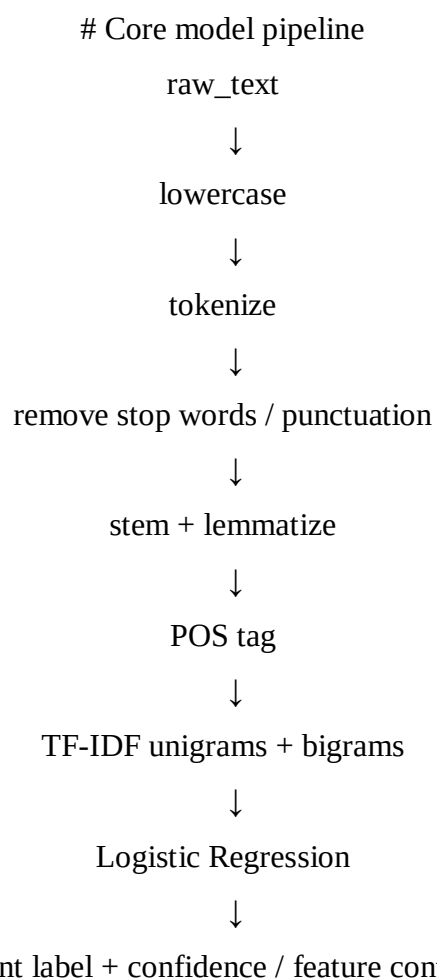
An important observation is that preprocessing does not always improve classification. Removing important words such as “not” can change sentence meaning and cause incorrect sentiment prediction. With additional time and resources, the system could be improved using a larger and more diverse dataset, multiple classification algorithms, better negation handling, word embeddings, transformer-based models and a graphical user interface for real-time interaction.

10. References

NLTK, pandas and scikit-learn software libraries used in the implementation.

- Bird, S., Klein, E., & Loper, E. Natural Language Processing with Python. O'Reilly Media.
- Manning, C. D., Raghavan, P., & Schütze, H. Introduction to Information Retrieval. Cambridge University Press.

Appendix A – Core Implementation Reference



The implementation dashboard also shows a polarity-lexicon contribution signal (LEX_NEG) in the sentiment explanation panel. The displayed negative example has confidence 1.00 and score -1.80 .

F. Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course/domain knowledge	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility, where applicable	5
Technical documentation & reflection	5
TOTAL	100

G. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem formulation			L3/L4	10
Application of knowledge			L3/L4	20
Solution / Design / Implementation			L4/L5/L6	20
Modern tool usage			L3/L4	10
Validation			L4/L5	15
Analysis & justification			L4/L5	15
Broader considerations			L4/L5	5

Documentation & reflection			L3/L4	5
----------------------------	--	--	-------	---

Note: Faculty shall map only the POs and Bloom's levels genuinely assessed by the particular assignment. Every assignment is not required to map to every PO.

H. Faculty Evaluation & Continuous Improvement

CO Attainment

Performance Level	Criterion
Level 3	$\geq 70\%$
Level 2	60–69%
Level 1	50–59%
Level 0	$< 50\%$

Target CO Attainment: _____

Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well:

Areas requiring improvement:

Corrective / Improvement Action:

Action to be implemented in the next offering:

Documents to be Maintained

The department/course faculty shall maintain:

1. Assignment question/problem statement
2. CO–PO and Bloom's mapping
3. Assessment rubric
4. Student submissions
5. Tool/simulation/experimental evidence, where applicable
6. Evaluated scripts/reports

7. Marks and rubric-wise assessment
8. CO attainment analysis
9. Sample student work representing different performance levels
10. Faculty analysis and continuous-improvement action